curso-js-html

js html

La integración de JavaScript en HTML es esencial para agregar interactividad y funcionalidad dinámica a sitios web. Existen varias formas de integrar JavaScript en HTML, pero la más común es a través de la etiqueta en el código HTML.

Una forma de integrar JavaScript directamente en HTML es colocando el código JavaScript dentro de la etiqueta en el código HTML. Por ejemplo:

```
<script>
// tu código JavaScript aquí
</script>
```

O poniendo su código js en un archivo independiente

```
<script src="miarchivo.js"></script>
```

Donde cargar el script

En un documento HTML, los scripts de JavaScript se pueden cargar en dos lugares principales: dentro del `<head>` o justo antes del cierre del `</body>`. La ubicación afecta el rendimiento y la funcionalidad.

1. **En el `<head>`:**
   - Tradicionalmente, los scripts se colocaban en el `<head>`. Sin embargo, cuando el navegador encuentra un script en el `<head>`, detiene la renderización de la página hasta que el script se descarga y ejecuta. Esto puede causar que la página se muestre más lentamente al usuario (bloqueo de renderizado).
   - Para mitigar esto, a menudo se usa el atributo `defer` o `async` cuando se coloca en el `<head>`.
2. **Justo antes del cierre del `</body>`:**
   - Colocar los scripts al final del body asegura que el HTML se cargue y renderice primero, lo que mejora la percepción de velocidad. Los scripts no bloquearán la renderización de la página.

Atributos importantes para scripts:

- `defer`: Indica que el script se ejecutará una vez que el documento HTML haya sido completamente analizado, pero antes del evento `DOMContentLoaded`. Los scripts con `defer` mantienen el orden de ejecución según su aparición en el documento.
- `async`: El script se ejecutará de manera asíncrona, es decir, tan pronto como esté disponible, sin esperar a que el analizador del HTML continúe. El orden de ejecución no está garantizado.

Recomendaciones:

- Si el script no depende del DOM y no es crítico, se puede usar `async` (en el `<head>` o en el `<body>`).
- Si el script depende del DOM o de otros scripts, y necesita ejecutarse en orden, es mejor usar `defer` en el `<head>` o colocarlo al final del `<body>` sin `defer` (aunque `defer` en el `<head>` es generalmente mejor para la carga porque no bloquea el analizador).

Ejemplos:

```html
<!DOCTYPE html>
<html>
<head>
  <title>Mi página</title>
  <script src="mi-script.js" defer></script>
</head>
<body>
  <!-- Contenido de la página -->
</body>
</html>
```

```html
<!DOCTYPE html>
<html>
<head>
  <title>Mi página</title>
</head>
<body>
  <!-- Contenido de la página -->

  <script src="mi-script.js"></script>
</body>
</html>
```

Evento onload

El evento **onload** en JavaScript se ejecuta cuando una página web (incluyendo todos sus recursos como imágenes, scripts y CSS) ha terminado de cargarse completamente. Se asocia comúnmente al objeto **window** o a elementos como `<body>`, y es útil para inicializar componentes que dependen de que todo el contenido esté listo.

```
window.onload = function() {
  // su código de inicialización aquí
  document.write(5 + 6);
};
```

```
document.addEventListener("DOMContentLoaded", () => {
  console.log("DOM listo, pero imágenes/scripts pueden seguir cargando");
});
```

Diferencia entre onload y DOMContentLoaded

Evento	Descripción
window.onload	Espera a que toda la página (imágenes, CSS, scripts) se cargue.
DOMContentLoaded	Se dispara cuando solo el DOM HTML está listo (sin esperar recursos externos). Más rápido.

Cargar datos asíncronos después del render:

```
window.addEventListener("load", async () => {
  const response = await fetch("datos.json");
  const datos = await response.json();
  console.log(datos);
});
```

Eventos

Asignar eventos:

```
<button onclick="document.write(5 + 6)">haz clic</button>
<input type="button" onclick="alert('Hello World!')" value="pruebame">
<inout type="button" id="miBoton" value="pincha aqui">
<input type="submit" value="Enviar">
<input type="submit">
```

Eventos dinámicos:

```
  document.querySelector("miBoton").onclick = function () {
    alert("¡Ouch! ¡Deja de pincharme!");
  };
```

Acceso a tags

```
<div class="w3-text-red" id="demo">
    <p id="mensaje">texto rojo</p>
</div>
<inout type="button" id="miBoton" value="pincha aqui">
```

```
document.querySelector("miBoton").onclick = function () {
    document.getElementById("demo").style.display = "none";
    document.getElementById("demo").innerHTML = "<p>My First JavaScript</p>";
    document.getElementById("mensaje").innerText = "Hello World";
    document.getElementById('demo').classList.add('w3-gray');
    document.getElementById('demo').classList.remove('w3-gray');
};

//referenciar todos los elementos de una clase, devuelve un arreglo
var array_el = document.querySelector(".miClase");
//referencias un elemento con id="foo"
var el = document.querySelector("#foo");
//
document.write(5 + 6);
window.alert(5 + 6);
//acceder a todos los parrafos, devuelve un arreglo
var div1Paras = div1.getElementsByTagName("p");
var num = div1Paras.length;
alert("Hay " + num + " <p> elementos en el elemento div1");
```

Los Selectores pueden ser muy útiles, ejemplo: Aquí, será retornado el input:

```
<div class="user-panel main">
    <input name="login" />
</div>
```

```
var el = document.querySelector("div.user-panel.main input[name='login']");
```

Objetos en js

> **curso-node-objetos**

Los objetos en JavaScript son colecciones de propiedades, donde cada propiedad es un par clave-valor. La clave es una cadena (o símbolo) y el valor puede ser cualquier tipo de dato (número, cadena, función, otro objeto, etc.).

Crear objetos

Forma 1 con Object:

```
var myCar = new Object();
myCar.make = "Ford";
myCar.model = "Mustang";
myCar.year = 1969;
```

Forma 2 usando iniciador de objeto:

```
var myCar = {
  make: "Ford",
  model: "Mustang",
  year: 1969,
};
```

Forma 3 con función constructora:

```
function Car(make, model, year) {
  this.make = make;
  this.model = model;
  this.year = year;
}
var mycar = new Car("Eagle", "Talon TSi", 1993);
```

Forma 4 con Object.create puedes heredar de otro objeto

```
const a = { a: 1 };
// acena de prototipos
// a ---> Object.prototype ---> null

const b = Object.create(a);
// cadena de prototipos
```

```
// b ---> a ---> Object.prototype ---> null
console.log(b.a); // 1 (inherited)

const c = Object.create(b);
// c ---> b ---> a ---> Object.prototype ---> null

const d = Object.create(null);
// d ---> null (d es un objeto que tiene null directamente como prototipo)
console.log(d.hasOwnProperty);
// undefined, porque d no hereda de Object.prototype
```

Otro ejemplo:

```
let padre = {
  saludo: function() {
    return "Hola";
  }
};

let hijo = Object.create(padre);
hijo.nombre = "Juan";

console.log(hijo.saludo()); // "Hola"
console.log(hijo.nombre); // "Juan"
```

Acceder a los atributos del objeto

Se accede a los elementos del objeto así:

```
myCar.make = "Ford";
myCar.model = "Mustang";
myCar.year = 1969;
//o asi
myCar["make"] = "Ford";
myCar["model"] = "Mustang";
myCar["year"] = 1969;
//Las propiedades no asignadas de un objeto son undefined (y no null).
//por ejemplo para el objeto anterior :
cosole.log(myCar.color); // undefined
```

desestructuración de objetos

```
const objeto = {
    propiedad: 'valor',
    otraPropiedad: 42
};

// Desesetructuración
const { propiedad } = objeto;
console.log(propiedad); // 'valor'

// Desestructuración con renombrado
const { propiedad: nuevoNombre, otraPropiedad: otroNuevoNombre } = objeto;
console.log(nuevoNombre); // 'valor'
console.log(otroNuevoNombre); // 42
console.log(propiedadVieja); // Error: propiedadVieja is not defined
```

ejemplo objetos

Un objeto persona:

```
const persona={
    id:0,
    nombre:"andres",
    carrera:"fisica",
    muestra:function(){
        console.log(`id:${this.id} nombre:${this.nombre}`)
    }
}

function fpersona(){
    this.id=0
    this.nombre="rosa"
    this.carrera="mate"
    this.muestra=function(){
        console.log(`id:${this.id} nombre:${this.nombre}`)
    }
}

persona3=new Object()
persona3.id=0
persona3.nombre="teresa"
persona3.carrera="algebra"
persona3.muestra=function(){
    console.log(`id:${this.id} nombre:${this.nombre}`)
}
```

```
persona.muestra()
const persona2=new fpersona()
persona2.muestra()
persona3.muestra()
```

Los objetos son asignados a otros objetos por referencia. Esto tiene el efecto que los vuelve mutables, Ejemplo:

```
//mutabilidad de objetos array
const array=[1,2,3,4,5]
const array2=array
array2[0]=45
console.log(array) //[ 45, 2, 3, 4, 5 ] , sin quererlo mutamos array
console.log(array2) //[ 45, 2, 3, 4, 5 ]
```

Clonar arreglos

```
const clon = [ ... arrayOriginal];
```

Clonar objetos

```
const original = { a: 1, b: 2 };
const clon = {  ... original };
```

Herencia

- `new Object()` o `{}` crea un objeto que hereda de `Object.prototype`.
- Para herencia de un objeto arbitrario, usa `Object.create(objetoPadre)`.
- Para herencia con funciones constructoras, establece el prototipo de la función constructora hija al prototipo del padre (usando `Object.create` y luego ajustando el `constructor`).
- Las clases de ES6 proporcionan una sintaxis más clara para la herencia.

Formas recomendadas de herencia

1. Object.create

```
const animal = {
  tipo: 'animal',
  respirar() {
    return 'respirando...';
  }
};

const perro = Object.create(animal); // Herencia real
perro.raza = 'labrador';
perro.ladrar = function() {
  return 'guau guau!';
};

console.log(Object.getPrototypeOf(perro) === animal); // true ✅
```

2. Clases ES6

```
class Animal {
  constructor(tipo) {
    this.tipo = tipo;
  }

  respirar() {
    return 'respirando...';
  }
}

class Perro extends Animal {
  constructor(raza) {
    super('mamifero');
    this.raza = raza;
  }

  ladrar() {
    return 'guau guau!';
  }
}

const miPerro = new Perro('labrador');
```

3. Funciones constructoras

```
function Animal(tipo) {
  this.tipo = tipo;
}

Animal.prototype.respirar = function() {
  return 'respirando...';
};

function Perro(raza) {
  Animal.call(this, 'mamifero');
  this.raza = raza;
}

// Herencia prototípica
Perro.prototype = Object.create(Animal.prototype);
Perro.prototype.constructor = Perro;

Perro.prototype.ladrar = function() {
  return 'guau guau!';
};
```

map

map

Sintaxis

```
let nuevoArray = array.map(function(elemento, indice, arreglo) {
  // Procesar el elemento y devolver un nuevo valor
});
```

Ejemplo

```
let users = [
    {firstName : "Susan", lastName: "Steward"},
    {firstName : "Daniel", lastName: "Longbottom"},
    {firstName : "Jacob", lastName: "Black"}
  ];

  let userFullnames = users.map(function(element){
      return `<li>${element.firstName} ${element.lastName}</li>`;
```

```
  })
   const salida=userFullnames.join('\n')
  console.log(userFullnames);
  console.log(salida)
```

Ejemplo 2 map:

```
let users = [
  {firstName: "Susan", lastName: "Steward", edad: 34},
  {firstName: "Daniel", lastName: "Longbottom", edad: 27},
  {firstName: "Jacob", lastName: "Black", edad: 18}
];

// Obtener array con lastname y edad
let lastNamesAndAges = users.map(user => ({
  lastName: user.lastName,
  edad: user.edad
}));

console.log(lastNamesAndAges);
// Resultado:
// [
//   {lastName: "Steward", edad: 34},
//   {lastName: "Longbottom", edad: 27},
//   {lastName: "Black", edad: 18}
// ]
```

funciones

curso-node-funciones

Existen 2 tipos de funciones: funciones y funciones flecha

```
//función
function suma(a,b){
    return a+b
}

//función flecha
const resta=(a,b)=>{
    return a-b
}

//funcion anonima
console.log((a,b)=>{
```

```
    return a*b
})
//función anonima
console.log(function(a,b){
    return a*b
})
```

Parámetros

```
function tablaMultiplicar(tabla, hasta) {
  for (let i = 0; i <= hasta; i++) {
    console.log(tabla, "x", i, "=", tabla * i);
  }
}

// Ejecución
tablaMultiplicar(5, 3); // "tabla" valdrá 5 y "hasta" valdrá 3

// Parámetros x default
function saludo(nombre = "Visitante", mensaje = "Hola") {
  console.log(mensaje + ", " + nombre);
}

saludo(); // "Hola, Visitante"
saludo("Ana"); // "Hola, Ana"
saludo("Carlos", "Bienvenido"); // "Bienvenido, Carlos"
```

Parámetros rest

```
//Parámetros rest
function sumar( ... numeros) {
  return numeros.reduce((acumulador, valorActual) => acumulador +
valorActual, 0);
}

console.log(sumar(1, 2)); // 3
console.log(sumar(5, 10, 15)); // 30
```

Desestructurar Parámetros

```
// Sin desestructuración
function mostrarUsuario(usuario) {
  console.log(usuario.nombre);
  console.log(usuario.edad);
  console.log(usuario.ciudad);
}

// CON desestructuración
function mostrarUsuario({ nombre, edad, ciudad }) {
  console.log(nombre);
  console.log(edad);
  console.log(ciudad);
}

// Uso
const usuario = {
  nombre: "Ana",
  edad: 30,
  ciudad: "Madrid"
};

mostrarUsuario(usuario);
// Output: Ana, 30, Madrid
```

formularios

Ejemplos de querySelctor

```
const header = document.querySelector('#header');
const botones = document.querySelectorAll('.boton');
const formulario = document.querySelector('form#principal');
const formulario = document.getElementById('principal');
```

acceso a formulario

```
<form id="miFormulario">
  <input type="text" name="usuario">
  <button type="submit">Enviar</button>
  <input type="submit">
</form>
```

```
const formulario = document.getElementById('miFormulario');
const formulario = document.querySelector('#miFormulario');
```

```
// Ejemplo: Escuchar evento de envío
formulario.addEventListener('submit', (e) => {
  e.preventDefault(); // Evita que se recargue la página
  //enviar formulario
  const formulario = document.getElementById('miFormulario');
  const formData = new FormData(formulario);
  const datosJSON = Object.fromEntries(formData.entries());
  console.log(datosJSON);
  console.log(JSON.stringify(datosJSON);)
  console.log('Formulario enviado');
});
```

acceder al formulario con forms

```
// Por ID (recomendado)
const formulario = document.forms.miFormulario; // o
document.forms['miFormulario']
// Por índice (si es el primer formulario de la página)
const primerFormulario = document.forms[0];
//o usando:
const formulario = document.querySelector('#miFormulario');
```

Acceder a los campos de un formulario

```
const formulario = document.getElementById('miFormulario');
// Por nombre (recomendado)
const inputUsuario = formulario.elements.usuario; // o
formulario.elements['usuario']
// Por índice
const primerCampo = formulario.elements[0];
// Valor del input
console.log(inputUsuario.value); // Muestra lo que el usuario escribió
```

querySelector

```
<form id="miFormulario">
    <input type="text" name="usuario" placeholder="Nombre de usuario">
    <input type="password" name="contrasena" placeholder="Contraseña">
</form>
```

```
// Acceder al input de usuario dentro de todo el documento
const inputUsuario = document.querySelector('input[name="usuario"]');
console.log(inputUsuario.value); // Muestra el valor del input

// Acceder al input de contraseña dentro de un formulario específico
const form = document.querySelector('#miFormulario');
const inputContrasena = form.querySelector('[name="contrasena"]');
console.log(inputContrasena.value);
```

getElementsByName

```
<form id="miFormulario">
    <input type="text" name="usuario" value="john_doe">
    <input type="email" name="email" value="john@example.com">
    <input type="radio" name="genero" value="masculino" checked>
    <input type="radio" name="genero" value="femenino">
</form>
```

```
// Acceder al input de usuario
const inputsUsuario = document.getElementsByName('usuario');
const inputUsuario = inputsUsuario[0]; // Primer elemento
console.log(inputUsuario.value); // "john_doe"

// Acceder a los radio buttons
const radiosGenero = document.getElementsByName('genero');
console.log(radiosGenero.length); // 2 (ambos radio buttons)

// Encontrar el radio button seleccionado
let generoSeleccionado;
for (let radio of radiosGenero) {
    if (radio.checked) {
        generoSeleccionado = radio.value;
        break;
    }
}
console.log(generoSeleccionado); // "masculino"
```

acceder x medio del id

```
<input type="text" id="usuario" placeholder="Nombre de usuario">
<input type="email" id="correo" value="ejemplo@mail.com">
```

```
// Usando el selector de ID (#)
const inputUsuario = document.querySelector('#usuario');
const inputCorreo = document.querySelector('#correo');
```

Acceder por variables globales window (no es una buena práctica)

un comportamiento histórico en los navegadores: cuando se asigna un `id` a un elemento HTML, ese `id` se convierte en una propiedad global (variable) en el objeto `window`. Sin embargo, esto no es una buena práctica y puede llevar a conflictos.

```
// Los elementos con ID se convierten en variables globales
console.log(usuario.value);    // Funciona si existe id="usuario"
console.log(correo.value);     // Funciona si existe id="correo"
```

obtener todos los campos de un formulario en un json

```
const formulario = document.getElementById('miFormulario');

formulario.addEventListener('submit', (e) => {
  e.preventDefault();

  // Paso 1: Crear un objeto FormData
  const formData = new FormData(formulario);
  // Paso 2: Convertir FormData a un objeto plano
  const datosJSON = Object.fromEntries(formData.entries());
  // Paso 3 (Opcional): Manejar checkboxes/radios no seleccionados
  // formData.has() se usa para verificar si un objeto FormData contiene una
clave (field) específica. Retorna true si la clave existe, false si no.
  if (!formData.has('suscribir')) {
    datosJSON.suscribir = false; // Si el checkbox no está marcado
  }
  console.log(datosJSON);
  // Ejemplo de salida:
  // { nombre: "Juan", email: "juan@example.com", pais: "mx", mensaje
```

ligas

IDE js en linea jsplayground
IDE como jsplayground pero mas avanzada
IDE avanzado de código en línea
probar css en linea
validar json

Excalidraw
caja de herramientas
generador de datos aleatorio
fakerjs para node